- Namespacing
- Autoloading
- Magic constants and introspection

Why OOP?

We're about to dive into quite a large topic, and if you're completely new to OOP, it might seem a little daunting. Although the focus of this chapter is mainly on the mechanisms of OOP, let me at least give a brief overview of its advantages.

Encapsulation – OOP allows data to be gathered together along with the functions that operate on them, and for those data and functions to be hidden from the rest of the program. By hiding data and code inside single units (objects) and controlling the flow of data between them, we can write safer and more robust software.

Modularity – decomposing software into a set of modules reduces the complexity of individual units of code. A problem is easier to understand (and solve) if it is broken into smaller problems. Modularity and encapsulation are closely related: the mechanisms of encapsulation enable modular design.

Reusability – OOP allows generalized, abstract units to define functionality that is then available to more specialized units through a mechanism called inheritance. When designed correctly, no repetition occurs.

Making Objects

What is an object? Simple, it's an instance of a class. And a class is a template for an object. (Don't you just love circular definitions?)

To be more specific, a class is the collection of related function definitions (called *methods* to distinguish them from non-object functions), property declarations, and initial state (state being the values of class and object properties at any given point